

Modüler Sistem Tasarımı

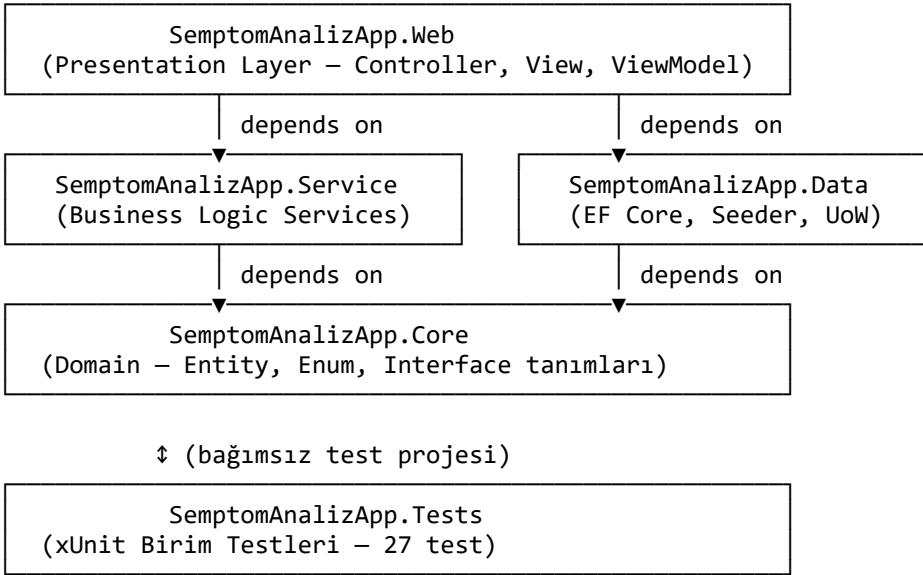
Proje: SemptomAnaliz

Hazırlayanlar: Arda Elibüyük, Furkan Taşdelen

Tarih: Nisan 2026

1. Genel Mimari

SemptomAnaliz, **Clean Architecture** (Temiz Mimari) prensiplerine dayalı 4 katmanlı modüler bir yapıya sahiptir. Her katman bağımsız bir .NET projesidir; yalnızca içe doğru bağımlılık (dependency inversion) kuralına uyulmuştur.



2. Katman Detayları

2.1 Core Katmanı — SemptomAnalizApp.Core

En iç katman; hiçbir dış projeye bağımlılığı yoktur.

Klasör	İçerik	Açıklama
Entities/	BaseEntity, Kullanici, SaglikProfili, AnalizOturumu, AnalizSonucu, Semptom, Hastalik, HastalikSemptom,	Domain nesneleri
Enums/	AnalizSemptomu, OlasiDurum, SemptomKatalog, AciliyetSeviyesi, BmiKategori, SemptomKategorisi, SemptomSiddeti	Sabit değer kümeleri
Interfaces/	IGenericRepository<T>, IUnitOfWork	Veri erişim sözleşmeleri

OOP Uygulaması: - BaseEntity abstract (soyut) sınıf → AnalizOturumu, Semptom, Hastalik vb. bu sınıftan **kalıtım** alır. - IGenericRepository<T> generic interface → **Encapsulation** ve **Dependency Inversion** sağlar.

2.2 Data Katmanı — SemptomAnalizApp.Data

Veritabanı erişim katmanı. Yalnızca Core'a bağımlıdır.

Dosya/Klasör	İçerik
AppDbContext.cs	EF Core DbContext; tüm DbSet tanımları
DbSeeder.cs	42 hastalık + semptom kataloğu + admin kullanıcı seed data
Migrations/	EF Core migration dosyaları (6 adet)
Repositories/GenericRepository.cs	IGenericRepository<T> implementasyonu
UnitOfWork.cs	Transaction yönetimi

OOP Uygulaması: - GenericRepository<T> sınıfı, IGenericRepository<T> interface'ini uygular → **Polymorphism** - Tüm veri erişimi protected readonly DbSet<T> ile → **Encapsulation**

2.3 Service Katmanı — SemptomAnalizApp.Service

İş mantığı katmanı. Core'a bağımlıdır; veri erişimi için Core'daki IAnalizDbContext sözleşmesini kullanır. Somut AppDbContext Web katmanında DI ile bağlanır.

Dosya	İçerik
Interfaces/IAnalizService.cs	Analiz motoru sözleşmesi + SemptomGirdisi record
Interfaces/I*Service.cs	BMI, imza, Bayes, aciliyet, tekrar, metin ve öneri servis sözleşmeleri
Services/AnalizMotoru.cs	Analiz akışını yöneten orkestrasyon servisi
Services/BayesianAnalizService.cs	Naive Bayes log-likelihood hesabı
Services/BmiService.cs	BMI ve BMI kategorisi hesabı
Services/AciliyetService.cs	Aciliyet skoru ve seviye hesabı
Services/TekrarAnalizService.cs	30 günlük tekrar imzası kontrolü
Services/AnalizMetinService.cs	Açıklama ve genel yorum üretimi
Services/GunlukOneriService.cs	Günlük öneri ve uyarı üretimi

OOP Uygulaması: - AnalizMotoru, IAnalizService interface'ini uygular → **Polymorphism** - Controller sadece IAnalizService'i tanır, implementasyonu bilmez → **Loose Coupling** - Algoritma parçaları küçük servislerde ayrıştırılmıştır → **Encapsulation**

Analiz Algoritması Özeti:

$$\begin{aligned} \log_posterior(D) = & \log(prior(D)) \\ & + \sum \log(P(S_i|D) / P(S_i|\neg D)) \quad [\text{seçili semptomlar}] \\ & + \sum 0.4 \times \log(P(S_i|D) / P(S_i|\neg D)) \quad [\text{yüksek ağırlıklı eksik semptomlar}] \\ & + \log(\text{yaş_modifier} \times \text{cinsiyet_modifier} \times \text{süre_modifier} \times \text{şiddet_modifier}) \end{aligned}$$

Sonuç → softmax normalizasyon → 0-100 göreceli benzerlik skoru

2.4 Web Katmanı — SemptomAnalizApp.Web

Sunum katmanı. Tüm projelere bağımlıdır.

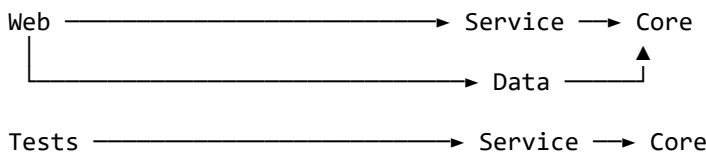
Klasör	İçerik
Controllers/	HomeController, AnalizController, HesapController, ProfilController, GecmisController, AdminController
ViewModels/	AnalizViewModels, HesapViewModels, ProfilViewModel, GecmisViewModel, DashboardViewModel
Views/	Razor View dosyaları (her controller için ayrı klasör)
Views/Shared/	_Layout.cshtml, _LoginPartial.cshtml
wwwroot/	CSS, JS, Bootstrap, jQuery kütüphaneleri
Program.cs	DI container, middleware pipeline, uygulama başlatma
appsettings.json	Bağlantı dizesi, seed şifre yapılandırması

2.5 Test Projesi — SemptomAnalizApp.Tests

Dosya	İçerik
AnalizMotoruTests.cs	27 xUnit birim testi

Test kapsamı: - HesaplaBmi — 7 senaryo (sıfır boy, null profil, tüm BMI kategorileri) - SkoraSeviyeAta — 8 eşik değeri senaryosu (Normal/İzle/Dikkat/Acil) - OlusturImza — 5 senaryo (uzunluk, küçük harf hex, sıra bağımsızlık, çakışma) - HesaplaAciliyetSkoru — 5 senaryo (boş girdi, kritik semptom, yaşlı, kronik, max 100)

3. Bağımlılık Grafiği



Her ok bağımlılık yönünü gösterir. Core hiçbir projeye bağımlı değildir.

4. Güvenlik Modülü

Program.cs middleware pipeline'ında yapılandırılmıştır:

Bileşen	Açıklama
ASP.NET Core Identity	Kullanıcı kaydı, giriş, bcrypt şifre hash
Cookie Authentication	14 günlük sliding expiration
CSRF Token	Tüm POST formlarında @Html.AntiForgeryToken()
Rate Limiter	Analiz endpoint'i: 10 istek/dakika
Security Headers	X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy
Authorization	[Authorize] attribute ile controller bazlı erişim kontrolü

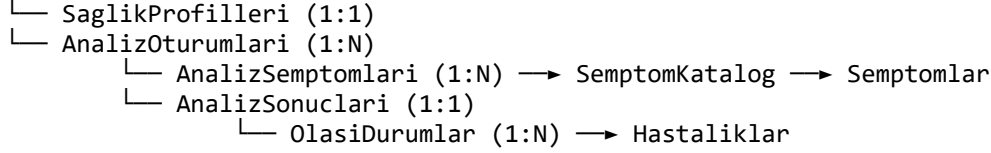
Bileşen**Açıklama**

Role-based Auth

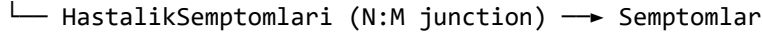
“Admin” ve “Kullanıcı” rolleri; Admin paneli yalnızca Admin’e açık

5. Veritabanı Şeması (Özet)

Kullanıcılar (ASP.NET Identity)



Hastalıklar



6. Klasör Yapısı

SemptomAnalizApp/	
├── docs/	# Proje belgeleri (bu klasör)
│ ├── GereksinimAnalizi.md	# Gereksinim analizi dokümanı
│ ├── ModulerTasarim.md	# Bu doküman
│ └── UML/	
│ ├── UseCaseDiagram.puml	# Use Case diyagramı (PlantUML)
│ └── ClassDiagram.puml	# Sınıf diyagramı (PlantUML)
├── src/	
│ ├── SemptomAnalizApp.Core/	# Domain katmanı
│ │ ├── Entities/	# Entity sınıfları (BaseEntity + türevleri)
│ │ ├── Enums/	# Enum tanımları
│ │ └── Interfaces/	# Repository ve servis arayüzleri
│ ├── SemptomAnalizApp.Data/	# Veri erişim katmanı
│ │ ├── Migrations/	# EF Core migration dosyaları
│ │ ├── Repositories/	# Generic Repository implementasyonu
│ │ ├── AppDbContext.cs	# EF Core DbContext
│ │ ├── DbSeeder.cs	# Seed data
│ │ └── UnitOfWork.cs	# Transaction yönetimi
│ ├── SemptomAnalizApp.Service/	# İş mantığı katmanı
│ │ ├── Interfaces/	# Servis arayüzleri
│ │ └── Services/	# AnalizMotoru ve küçük analiz servisleri
│ ├── SemptomAnalizApp.Web/	# Sunum katmanı
│ │ ├── Controllers/	# MVC Controller'lar (6 adet)
│ │ ├── ViewModels/	# ViewModel sınıfları
│ │ ├── Views/	# Razor View'lar
│ │ ├── wwwroot/	# Statik dosyalar (CSS, JS, kütüphaneler)
│ │ ├── Program.cs	# Uygulama başlatma ve DI
│ │ └── appsettings.json	# Yapılandırma
│ ├── SemptomAnalizApp.Tests/	# Birim testler
│ │ └── AnalizMotoruTests.cs	# 27 xUnit testi
├── SemptomAnalizApp.sln	# Visual Studio solution dosyası
├── README.md	# Proje genel açıklaması
└── .gitignore	# Git yoksayma kuralları

*Bu doküman SemptomAnaliz projesinin modüler sistem tasarımını açıklamaktadır.
Son güncelleme: Nisan 2026*